**Washing Station**

Read the guideline before starting. You own a car washing station. Washing cost calculation takes a lot of time, and you decide to automate this calculation. The washing cost will depend on car comfort class, car cleanness degree, wash station average rating and wash station distance from the center of the city. Create class Car, its __init__ method takes and stores 3 arguments:

1. comfort_class - comfort class of a car, from 1 to 7
2. clean_mark - car cleanness mark, from very dirty - 1 to absolutely clean - 10
3. brand - brand of the car

Create class CarWashStation, its __init__ method takes and stores 4 arguments:

1. distance_from_city_center - how far station from the city center, from 1.0 to 10.0
2. clean_power - clean_mark to which this car wash station washes (yes, not all stations can clean your car completely)
3. average_rating - average rating of the station, from 1.0 to 5.0, rounded to 1 decimal
4. count_of_ratings - number of people who rated

CarWashStation should have such methods:

1. serve_cars - method, that takes a list of Car's, washes only cars with clean_mark < clean_power of wash station and returns income of CarWashStation for serving this list of Car's, rounded to 1 decimal:

```
bmw = Car(comfort_class=3, clean_mark=3, brand='BMW')

audi = Car(comfort_class=4, clean_mark=9, brand='Audi')

print(bmw.clean_mark)  # 3

wash_station = CarWashStation(

  distance_from_city_center=5,

  clean_power=6,

  average_rating=3.5,

  count_of_ratings=6

)

income = wash_station.serve_cars([bmw, audi])
```

```
print(income)  # 6.3

print(bmw.clean_mark)  # 6
```

So, only bmw was washed, because audi.clean_mark > wash_station.clean_power, and bmw.clean_mark has changed, because we washed it. If audi.clean_mark was below wash_station.clean_power then audi would have been washed as well and the income would have raised:

```
bmw = Car(comfort_class=3, clean_mark=3, brand='BMW')

audi = Car(comfort_class=4, clean_mark=2, brand='Audi')

print(bmw.clean_mark)  # 3

print(audi.clean_mark) # 2

wash_station = CarWashStation(

  distance_from_city_center=5,

  clean_power=6,

  average_rating=3.5,

  count_of_ratings=6

)

income = wash_station.serve_cars([bmw, audi])

print(income)  # 17.5

print(bmw.clean_mark)  # 6

print(audi.clean_mark) # 6
```

2. calculate_washing_price - method, that calculates cost for a single car wash, cost is calculated as: car's comfort class * difference between wash station's clean power and car's clean mark * car wash station rating / car wash station distance to the center of the city, returns number rounded to 1 decimal;
3. wash_single_car - method, that washes a single car, so it should have clean_mark equals wash station's clean_power, if wash_station.clean_power is greater than car.clean_mark;
4. rate_service - method that adds a single rate to the wash station, and based on this single rate average_rating and count_of_ratings should be changed:

```
wash_station = CarWashStation(
```

```
    distance_from_city_center=6,
    clean_power=8,
    average_rating=3.9,
    count_of_ratings=11
)
print(wash_station.average_rating)    # 3.9
print(wash_station.count_of_ratings)  # 11
wash_station.rate_service(5)
print(wash_station.average_rating)    # 4.0
print(wash_station.count_of_ratings)  # 12
```

You can add own methods if you need. Example:

```
bmw = Car(3, 3, 'BMW')
audi = Car(4, 9, 'Audi')
mercedes = Car(7, 1, 'Mercedes')
ws = CarWashStation(6, 8, 3.9, 11)
income = ws.serve_cars([
    bmw,
    audi,
    mercedes
])
income == 41.7
bmw.clean_mark == 8
audi.clean_mark == 9
mercedes.clean_mark == 8
# audi wasn't washed
# all other cars are washed to '8'
ford = Car(2, 1, 'Ford')
wash_cost = ws.calculate_washing_price(ford)
```

*# only calculating cost, not washing*

wash_cost == 9.1

ford.clean_mark == 1

ws.rate_service(5)

ws.count_of_ratings == 12

ws.average_rating == 4.0